

Transformer

2026 年 9 月 28 日

1 引言

第一次看 Transformer 的框架图时，我能记住 Encoder、Decoder、Attention 这些名字，却很难把它们连成一个真正能计算的模型。尤其是那条注意力公式：为什么三个矩阵乘来乘去，就能让模型利用上下文？既然注意力已经处理了输入，为什么还要把原输入加回来？右边已经画了 Outputs，模型究竟是在读取答案，还是在生成答案？

后来我发现，困难不在于术语太多，而在于每一步的输入、输出和目的没有接上。因此，这篇笔记不从模块名单开始，而是沿着一条数据流展开：一句话如何变成矩阵，矩阵如何交换信息，最后如何得到下一个 token 的概率。中间会手算一次注意力，也会单独解释位置编码、多头、残差与归一化的设计动机。

本文讨论的是 *Attention Is All You Need* 中的原始编码器—解码器 Transformer，采用图中“先 Add、后 Norm”的结构。为了看清矩阵运算，手算部分使用四个 token、四维输入；完整问答部分改用八个输入 token、十二维表示。两者都是教学尺寸，不是原论文的模型配置。本文不展开优化器调度、分词算法和高效推理工程，先把基础计算链条接通。本文主要参考了原论文《Attention Is All You Need》、《动手学深度学习》的相关章节与 PyTorch 文档，侧重于直观理解，省略了部分纯计算性的细节，并在其中附上了笔者自己的理解，如有错误，还请指正。

2 序列任务与注意力

2.1 序列任务的困难

机器翻译、问答等任务，都需要根据一个序列产生另一个序列。例如输入“谁是第一个登上月球的人？”，输出一个人名。仅仅识别每个词还不够，因为“第一个”“月球”“人”共同约束了问题的含义。模型需要利用不同位置之间的依赖。

一种自然的处理方式是顺序读取。循环神经网络（RNN）在第 i 个位置计算

$$\mathbf{h}_i = f_{\theta}(\mathbf{h}_{i-1}, \mathbf{x}_i). \quad (2.1)$$

这里 $\mathbf{h}_i$ 是读到当前位置时的内部状态。由于计算 $\mathbf{h}_i$ 需要先得到 $\mathbf{h}_{i-1}$ ，同一个序列内部存在串行依赖；相隔很远的位置之间，也往往需要经过多步状态传递。LSTM 等结构改善了记忆问题，但没有消除这种逐位置计算的依赖。

注意力提供了另一种办法：处理当前位置时，直接对允许访问的位置打分，再汇总其中的信息。它在 Transformer 之前就已经被用于神经机器翻译；Transformer 的关键并不是首次发明注意力，而是用注意力承担主要的序列信息交互，不再依赖原来的循环结构。

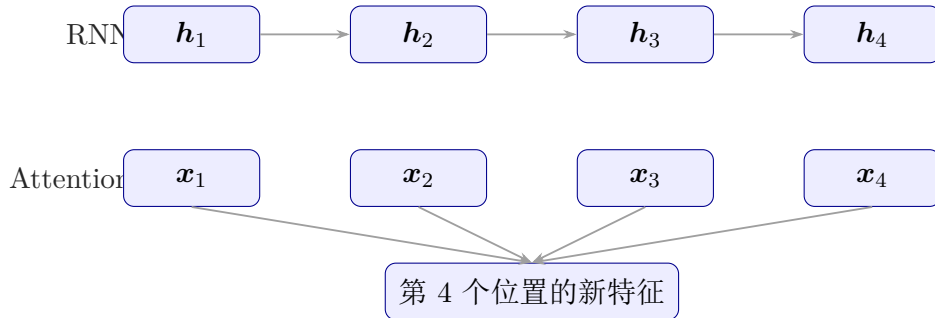


图 1: 上排存在逐步状态依赖；下排仅画出第 4 个位置直接汇总各位置信息的过程。其他查询位置也会各自进行汇总。

这并不意味着 Transformer 在任何条件下都更快，也不意味着长距离依赖已经被彻底解决。完整注意力需要考虑大量位置配对，序列很长时仍然昂贵。它提供的是更直接的信息通路，以及同一层内更适合并行的计算

方式：层与层之间仍要顺序执行，自回归生成也仍要逐步产生新 token。

2.2 输入与输出

原始 Transformer 采用 sequence-to-sequence (seq2seq) 形式：Encoder 把输入序列转换成一串连续向量；Decoder 读取这些向量以及已知的输出前缀，预测下一个 token。输入序列与输出序列不必一样长。

“seq2seq”描述输入、输出之间的组织方式，“注意力”描述内部如何汇总信息，两者并不冲突。本文先拆开计算，再回到第 7 节的完整框图。所有向量默认按行摆放； n 表示输入长度， t 表示当前输出前缀长度， d 表示每个位置的表示维度。暂时不写 batch 维度，以免它遮住核心计算。

3 输入表示

3.1 Token 与词嵌入

Token 是分词器使用的基本单位，未必等于完整单词，也可能是字或子词。为方便手算，这里把 I love deep learning 当作四个 token。词表为它们分配编号；编号只负责索引，不表示语义大小。

Embedding（嵌入）是一张可训练的查找表。设词表为 $\mathcal{V}$ ，则嵌入表

$$\mathbf{B} \in \mathbb{R}^{|\mathcal{V}| \times d} \quad (3.1)$$

为每个 token 存放一个 d 维向量。输入一个编号，就取出对应的一行；这不是把编号本身当成数值特征参与语义计算。

例如，假设查表结果如表 1 所示。所有数字都是人为选择的演示值，不代表真实模型中的词义。

按句子顺序把取出的四行叠起来，就得到当前句子的嵌入矩阵

$$\mathbf{E}_s = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}. \quad (3.2)$$

表 1: 四个 token 的示意嵌入；编号从零开始，但此处只列出部分词表

Token	编号	查表得到的向量
I	12	[1, 0, 1, 0]
love	37	[0, 1, 1, 0]
deep	81	[0, 2, 0, 1]
learning	96	[1, 1, 0, 1]

注意区分： $\mathbf{B}$ 是整张词表的参数； $\mathbf{E}_s$ 是当前这句话查表得到的中间结果。一个位置对应一行，一个特征对应一列。

这些向量并不是人工定义好的“词义坐标”。从头训练时可以随机初始化，随后通过最终任务的损失与其他参数一起调整。每个维度通常没有预先规定的人类语义。参数固定时，同一个 token 的初始查表结果相同；经过后面的上下文计算后，它在不同句子中的表示才可以不同。

3.2 位置编码

只有词的向量，还缺少顺序信息。“人咬狗”和“狗咬人”使用同一组词，但顺序改变了含义。不带位置机制、且采用相同全连接访问规则的自注意力，不能仅凭数组的行号自动获得这种顺序含义。

原始 Transformer 生成一个与嵌入矩阵同形状的位置矩阵 $\mathbf{P}$ ，再相加。严格写出原论文中的嵌入缩放，有

$$\mathbf{X}^{(0)} = \sqrt{d} \mathbf{E}_s + \mathbf{P}, \quad \mathbf{E}_s, \mathbf{P}, \mathbf{X}^{(0)} \in \mathbb{R}^{n \times d}. \quad (3.3)$$

例如，八个 token、每个十二维时，相加前后都是 8×12 ，不是 8×13 。两个矩阵是在相同位置上逐元素相加，不是首尾拼接。

原始的正弦位置编码为

$$\begin{aligned} P_{p,2j} &= \sin\left(\frac{p}{10000^{2j/d}}\right), \\ P_{p,2j+1} &= \cos\left(\frac{p}{10000^{2j/d}}\right), \end{aligned} \quad (3.4)$$

其中位置 p 从零开始；对偶数 d ，有 $j = 0, \dots, d/2 - 1$ 。不同维度使用不同频率，因此一个位置对应一整组有规律的数，而不是单独附加一个整数编号。

3.3 相加的唯一性

会，任意选择的两组数字确实可能发生这种碰撞。暂时将式 (3.3) 的共同缩放吸收入 e 的记号，取

$$\begin{aligned} e(a) &= [0.1, 0.2, 0], & e(b) &= [0, 0.1, -0.1], \\ p_1 &= [0.1, 0.1, 0.1], & p_2 &= [0, 0, 0]. \end{aligned}$$

就有

$$e(a) + p_2 = e(b) + p_1 = [0.1, 0.2, 0]. \quad (3.5)$$

单靠合成后的这个向量，无法区分这两个组合。不能用“网络会理解”来回避这一点：相加不是对任意数字都保证可逆、保证唯一的编码。

实际设计利用的是有结构的位置编码和可训练的嵌入。模型从训练开始就在联合表示上工作，可以学习适合任务的内容与位置组合；它并不要求嵌入原来的每个数都保持不变。式 (3.5) 说明了这种设计没有无条件的唯一性保证，但并不说明所有相加都会失去身份信息。位置编码本身也不是事实知识，更不会自动告诉模型答案。

4 单头自注意力

4.1 注意力层的输出

注意力层的目的不是给整句话选一个“最重要的词”，而是为每个查询位置生成一份上下文汇总。第 i 个位置有自己的分配比例，决定从不同位置分别取多少信息。这份比例可以随输入内容改变。

为把计算压缩到纸上，下面直接指定当前注意力子层的输入为

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix}. \quad (4.1)$$

四行仍对应 I/love/deep/learning。这里专门设置了一个手算输入；不要把它误认为式 (3.2) 添加真实位置编码后的计算结果。手算段不再额外加位置编码，也省略投影偏置。

4.2 Query、Key、Value 的来源

同一个位置要参与两类工作：一类是决定“如何匹配和分配”，另一类是提供“真正被汇总的内容”。Query 与 Key 参与匹配，Value 参与最后的汇总。三者由输入经过不同的可训练线性投影得到：

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V. \quad (4.2)$$

这里 $\mathbf{X}$ 不是 $\mathbf{Q}$ ，而是产生 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 的原材料。

假设选择下面三套 4×2 的权重：

$$\mathbf{W}_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & -1 \end{bmatrix}, \quad \mathbf{W}_K = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{W}_V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}. \quad (4.3)$$

“乘权重矩阵”就是按系数组合原来的特征。对于 $\mathbf{x} = [a, b, c, d]$ ，这三个矩阵分别实现

$$\mathbf{q} = [a, b + c - d], \quad \mathbf{k} = [b, c], \quad \mathbf{v} = [a + c, b + d]. \quad (4.4)$$

真实模型不会固定使用这几条规则，而是在训练中调整系数。对所有行应

用同一套规则，得到

$$\mathbf{Q} = \begin{bmatrix} 1 & 1 \\ 0 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 0 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \\ 0 & 3 \\ 1 & 2 \end{bmatrix}. \quad (4.5)$$

三者形状都是 4×2 。到这里还没有交换 token 之间的信息，只是每一行分别做了特征变换。

4.3 分数矩阵

先只看第四个位置 learning。它的 Query 是 $\mathbf{q}_4 = [1, 0]$ ，与各 Key 点积：

$$\begin{aligned} \mathbf{q}_4 \mathbf{k}_1^\top &= 0, & \mathbf{q}_4 \mathbf{k}_2^\top &= 1, \\ \mathbf{q}_4 \mathbf{k}_3^\top &= 2, & \mathbf{q}_4 \mathbf{k}_4^\top &= 1. \end{aligned} \quad (4.6)$$

这得到 $[0, 1, 2, 1]$ ，分别表示该位置对四个来源的匹配分数。这里包括自己；普通自注意力通常允许查看自己，但自己的分数并不保证最大。

其他查询位置也进行相同计算。将它们集中成矩阵乘法，就是

$$\mathbf{S} = \mathbf{QK}^\top = \begin{bmatrix} 1 & 2 & 2 & 1 \\ 2 & 2 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 \end{bmatrix}. \quad (4.7)$$

转置的作用是把每个 Key 从一行变成一列，使矩阵乘法实现所有 Query 与所有 Key 的点积：

$$(4 \times 2)(2 \times 4) \longrightarrow 4 \times 4.$$

因此 S_{ij} 的行 i 是接收信息的位置，列 j 是候选来源。 $\mathbf{S}$ 的列已经不是原始特征维度，而是序列中的位置。

4.4 缩放与 softmax

这个头的 Query 与 Key 都是二维，所以 $d_k = 2$ 。先把每个分数除以 $\sqrt{2}$ ；矩阵形状不变。第四行变成

$$[0, 1, 2, 1]/\sqrt{2} \approx [0, 0.707, 1.414, 0.707].$$

为什么要缩放？在一个简化假设下，Query 和 Key 的各分量独立、均值为零、方差为一，则

$$\text{Var}\left(\sum_{r=1}^{d_k} q_r k_r\right) = d_k. \quad (4.8)$$

点积的标准差随 $\sqrt{d_k}$ 增长。除以这个量可以控制初始分数尺度，减少 softmax 过度集中的风险；这是一种尺度设计，并不是声称训练后的分量永远满足上述独立假设。

接下来对每一行分别做 softmax：

$$A_{ij} = \frac{\exp(S_{ij}/\sqrt{d_k})}{\sum_{r=1}^n \exp(S_{ir}/\sqrt{d_k})}. \quad (4.9)$$

第四行先取指数约为 $[1, 2.028, 4.113, 2.028]$ ，再除以这一行的总和，得到

$$\mathbf{a}_4 \approx [0.109, 0.221, 0.449, 0.221]. \quad (4.10)$$

四个数相加为一，表示当前查询位置如何分配汇总比例。所有行一起算得

$$\mathbf{A} = \text{softmax}(\mathbf{S}/\sqrt{2}) \approx \begin{bmatrix} 0.165 & 0.335 & 0.335 & 0.165 \\ 0.402 & 0.402 & 0.098 & 0.098 \\ 0.335 & 0.335 & 0.165 & 0.165 \\ 0.109 & 0.221 & 0.449 & 0.221 \end{bmatrix}. \quad (4.11)$$

$\mathbf{A}$ 叫作注意力权重矩阵。它不是统计学的相关系数矩阵，不必对称，也不是训练前就固定的一张表；它随当前输入与参数而变化。仅凭权重较大，也不能把它当成模型作答理由的完整解释。

4.5 加权求和输出

第四个位置的输出是四个 Value 的加权和：

$$\begin{aligned}
 z_4 &= \sum_{j=1}^4 A_{4j} \mathbf{v}_j \\
 &\approx 0.109[2, 0] + 0.221[1, 1] + 0.449[0, 3] + 0.221[1, 2] \\
 &\approx [0.660, 2.009].
 \end{aligned} \tag{4.12}$$

一个数乘向量，就是把向量每个元素都乘上该数；向量相加，则把相同维度的元素相加。上式第一维是 $0.109 \times 2 + 0.221 \times 1 + 0.449 \times 0 + 0.221 \times 1$ 。这些数只是保留三位小数后的展示；最终结果按未舍入权重计算。

这和平均数很像，但不是每个来源都占 $1/4$ ，而是每个查询位置根据内容决定自己的权重。对所有位置一次完成汇总，就是

$$\mathbf{Z} = \mathbf{A}\mathbf{V} \approx \begin{bmatrix} 0.830 & 1.670 \\ 1.304 & 0.891 \\ 1.170 & 1.160 \\ 0.660 & 2.009 \end{bmatrix} \in \mathbb{R}^{4 \times 2}. \tag{4.13}$$

$\mathbf{A}$ 是“怎么取”的规则， $\mathbf{Z}$ 是“实际取到了什么”的特征。前者每行四个权重，后者每行两个输出特征，不能混为一谈。

现在把三步连起来，得到熟悉的公式：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \tag{4.14}$$

它不是“算完相关性就结束”，而是先匹配，再归一化分配，最后用分配结果汇总内容。

Scaled Dot-Product Attention

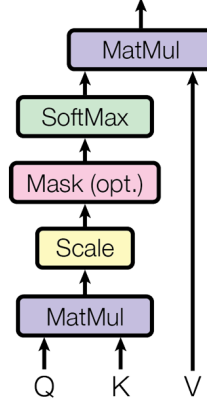


图 2: 缩放点积注意力 (原论文 Figure 2 左)。\$Q\$ 与 \$K\$ 先做矩阵乘法, 缩放后按需掩码, 再经 softmax 得到权重, 最后与 \$V\$ 相乘。

5 多头注意力

5.1 头的投影

恢复到 $\mathbf{X} \in \mathbb{R}^{8 \times 12}$ 的输入尺寸。一个头只有一套注意力分配方式: 对某个来源给出权重 a , 就会把该来源的整个 Value 向量乘上同一个 a 。多头允许同时计算多套分配与内容表示, 再合并结果。

设有三个头, 每个头的 Query、Key、Value 都是四维。第 h 个头计算

$$\mathbf{Q}_h = \mathbf{XW}_Q^{(h)}, \quad \mathbf{K}_h = \mathbf{XW}_K^{(h)}, \quad \mathbf{V}_h = \mathbf{XW}_V^{(h)}, \quad (5.1)$$

其中每个权重矩阵都是 12×4 。因此每个头都能组合完整的十二维输入, 并不是先截取原向量的前四维、中间四维和后四维。它们各自得到 8×8 的权重矩阵, 以及 8×4 的输出 $\mathbf{Z}_h$ 。

各头不必独立保存全部输入信息, 也没有预先规定“一个头负责主谓、一个头负责指代”。它们具有不同参数, 但通过最终任务的同一个损失联合学习; 不是分别训练三个独立模型。

一般写成

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\mathbf{Z}_1, \dots, \mathbf{Z}_H)\mathbf{W}_O, \quad \mathbf{W}_O \in \mathbb{R}^{Hd_v \times d}. \quad (5.2)$$

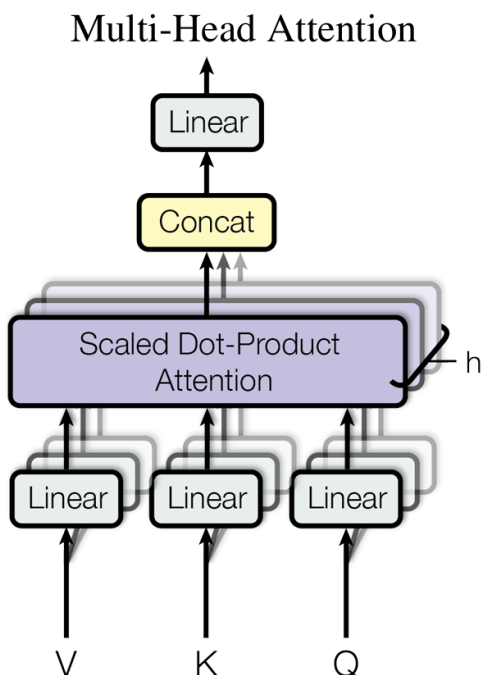


图 3: 多头注意力 (原论文 Figure 2 右)。各头读取相同的完整输入, 各自完成一次缩放点积注意力, 拼接后再经过一次输出投影。

实现中可以把多个投影合成一次大矩阵乘法, 再把投影后的结果拆成头。例如

$$\mathbf{X}[\mathbf{W}_Q^{(1)} \mid \mathbf{W}_Q^{(2)} \mid \mathbf{W}_Q^{(3)}] = [\mathbf{Q}_1 \mid \mathbf{Q}_2 \mid \mathbf{Q}_3].$$

所谓“拆头”常常发生在这一步, 而不是在原始输入上直接切掉部分信息。

5.2 输出维度

各头的内部宽度并不必须是四。完全可以让每个头输出 8×5 , 三个头拼成 8×15 , 再选择

$$\mathbf{W}_O \in \mathbb{R}^{15 \times 12}, \quad (8 \times 15)(15 \times 12) \longrightarrow 8 \times 12. \quad (5.3)$$

回到十二维, 是为了与网络主路径的宽度一致, 方便残差相加和逐层连接。常见的 $d_v = d/H$ 只是控制内部总宽度、计算量和参数量的一种选择, 不是不可改变的数学定律。

八行之所以保留，是因为这里有八个 Query，需要分别输出八个向量。行数由查询位置数决定，列数则由特征设计和输出投影决定。

6 Add、Norm 与 FFN

6.1 残差连接

暂时不考虑归一化，用 F_θ 表示注意力或 FFN 这样的可训练模块。没有残差时，模块输出直接作为结果；有残差时，再加上该模块的输入：

$$\text{无残差: } \hat{\mathbf{y}} = F_\theta(\mathbf{x}), \quad \text{有残差: } \hat{\mathbf{y}} = \mathbf{x} + F_\theta(\mathbf{x}). \quad (6.1)$$

这确实把输入分成了两路，但不意味着处理效果被减半，也不是先训练好一个 F_θ ，最后临时加上 $\mathbf{x}$ 。从训练开始，损失评判的就是包含加法的整个计算图。残差结构改变了变换的参数化方式。

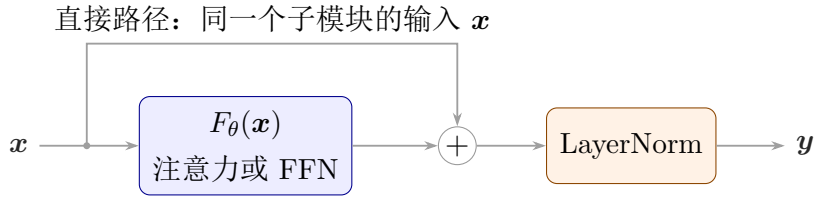


图 4: 原始 Transformer 子模块的计算：先相加，再归一化。跳过分支不是从每一层都回接最初的 Embedding。

例如，我们希望实现 $T(x) = 3x$ ，分支取 $F_\theta(x) = \theta x$ 。没有残差时需要 $\theta = 3$ ；有残差时只需要 $\theta = 2$ ，因为 $x + 2x = 3x$ 。目标没有改变，达到目标所需的参数改变了。

取训练样本 $x = 2, y^* = 6$ ，有残差时真正计算的损失是

$$L(\theta) = (2 + 2\theta - 6)^2, \quad \frac{dL}{d\theta} = 4(2 + 2\theta - 6). \quad (6.2)$$

最优参数是 $\theta = 2$ ，而不是先让分支拟合 6、再加出错误的 8。固定的直接路径没有待学参数，却仍然影响最终预测和梯度所依据的损失。

如果用 $T(\mathbf{x})$ 表示某个理想变换，则 $F_\theta(\mathbf{x})$ 可以被理解为学习 $T(\mathbf{x}) - \mathbf{x}$ 。但中间层并没有一份已知的 $T(\mathbf{x})$ 供我们直接相减：训练数据通常只给最终

答案，不会规定每一层每个特征的正确数值。网络要学习的是“当前输入需要怎样的更新”，不是学习减法运算。

残差分支的输出不是损失，也没有被要求始终趋近于零。

例如从 2 变到 6，更新量应该是 4。只在某个子层暂时不需要改变输入时，零更新才合适。上面的“学习差值”是代数解释；分支的实际表达能力和优化过程仍然决定能否拟合目标，不是任意结构都可以无条件学会任意变换。

6.2 直接路径与反向传播

先用一维说明，设 $y = x + F_\theta(x)$ ，后面的损失为 L 。链式法则给出

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} (1 + F'_\theta(x)). \quad (6.3)$$

没有残差时，括号里只有 $F'_\theta(x)$ 。例如这个导数是 0.01，后一层传来的梯度是 1，则两种情况分别得到 0.01 和 1.01。这个例子解释了为什么梯度不必完全穿过复杂分支才能传回前面；它不是对所有输入、所有层都不消失或不爆炸的保证。

多维情况对应 $\mathbf{I} + \mathbf{J}_F$ ，其中 $\mathbf{J}_F$ 是分支的雅可比矩阵。原始 Transformer 后面还接 LayerNorm，梯度也要经过它，不能把整层视为完全不变的恒等通道。只把分支设成零，原图的子层仍输出 $\text{LN}(\mathbf{x})$ ，而不是严格的 $\mathbf{x}$ 。

6.3 LayerNorm 的归一化对象

先做

$$\mathbf{U} = \text{MHA}(\mathbf{X}), \quad \mathbf{Z} = \mathbf{X} + \mathbf{U}, \quad \mathbf{Y} = \text{LN}(\mathbf{Z}). \quad (6.4)$$

把前两项代入，才得到 $\mathbf{Y} = \text{LN}(\mathbf{X} + \text{MHA}(\mathbf{X}))$ 。LN 实际只接收一个矩阵：相加后的 $\mathbf{Z}$ 。不是要求 LN 的括号里必须有两项。

对某一行 $\mathbf{z} = [z_1, \dots, z_d]$ ，计算

$$\mu = \frac{1}{d} \sum_{j=1}^d z_j, \quad \sigma^2 = \frac{1}{d} \sum_{j=1}^d (z_j - \mu)^2, \quad (6.5)$$

再逐维得到

$$\text{LN}(\mathbf{z})_j = \gamma_j \frac{z_j - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_j. \quad (6.6)$$

ϵ 是避免数值问题的小常数； γ, β 是可训练的缩放与平移参数。每个 token 分别沿自己的特征维度计算统计量；它不把整句话混在一起平均，也不是沿 batch 计算。

例如 $\mathbf{z} = [1, 2, 3, 4]$ 的均值为 2.5、方差为 1.25。暂取 $\gamma = \mathbf{1}, \beta = \mathbf{0}$ 并忽略很小的 ϵ ，得到

$$\text{LN}(\mathbf{z}) \approx [-1.342, -0.447, 0.447, 1.342].$$

若整体换成 $[100, 200, 300, 400]$ ，标准化部分近似相同。它控制的是数值尺度，不是把特征“净化”为正确答案；这种处理也不是无损的。加上可训练的缩放和平移后，最终输出不必保持均值为零、方差为一。

尺度为什么值得管？忽略偏置，若输入整体放大 c 倍，投影的 $\mathbf{Q}, \mathbf{K}$ 也随之放大，点积变成 $c^2 \mathbf{Q} \mathbf{K}^\top$ ，softmax 的集中程度就可能明显改变。LN 有助于让后续层面对较受控的数值尺度，并改善深层网络的训练性质。

原图使用先相加、再归一化的 Post-LN。把 LN 放到分支前的 Pre-LN 是另一种结构，例如 $\mathbf{y} = \mathbf{x} + F_\theta(\text{LN}(\mathbf{x}))$ ；两者不是等价的括号挪动，训练性质也不同。本文只追踪原图，不在这里展开变体。

6.4 FFN 与非线性

注意力负责按内容汇总不同位置的信息；FFN 则对每个位置已经得到的特征继续做变换。原始结构为

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x} \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2. \quad (6.7)$$

例如可选 $12 \rightarrow 48 \rightarrow 12$ 的宽度，其中 $\text{ReLU}(x) = \max(x, 0)$ 是逐元素取正部。中间扩展提供更多候选特征，非线性使它不再只是一次固定的线性组合。同一层的各位置共用 FFN 参数，但分别计算。

一个极小的例子就能展示用意。让输入为一个数 x ，两次线性变换和一次 ReLU 实现

$$x \longrightarrow [x, -x] \longrightarrow [\max(x, 0), \max(-x, 0)] \longrightarrow |x|. \quad (6.8)$$

输入 -2 ，依次得到 $[-2, 2]$ 、 $[0, 2]$ ，最后得到 2 。这正是一次 $1 \rightarrow 2 \rightarrow 1$ 的 FFN：升维不是为了“洗干净”数据，而是和非线性一起构造不同的输入响应。

如果拿掉非线性、暂不写偏置，则

$$(\mathbf{x}\mathbf{W}_1)\mathbf{W}_2 = \mathbf{x}(\mathbf{W}_1\mathbf{W}_2),$$

两次线性变换可以合成一次；包含偏置时也仍能合并成一个仿射变换。单纯升维再降维不会自动增加这样的表达能力。

注意力自身也包含 softmax 等非线性，所以 FFN 不是全模型唯一的非线性来源。它的特别之处是提供逐位置的、可学习的非线性特征变换，而不在这一步重新混合 token 位置。

6.5 完整的 Encoder Block

忽略 dropout，原始 Encoder 一层只需写成

$$\begin{aligned} \mathbf{H} &= \text{LN}_1(\mathbf{X} + \text{MHA}(\mathbf{X})), \\ \mathbf{Y} &= \text{LN}_2(\mathbf{H} + \text{FFN}(\mathbf{H})). \end{aligned} \quad (6.9)$$

每个主要子模块都配一组 Add 与 Norm，所以一层有两组。第二次 Add 加回的是 FFN 的输入 $\mathbf{H}$ ，不是最初的 $\mathbf{X}$ 。两个 LN 有各自的可训练参数，但输出形状都不变。

原论文训练时还在分支输出等位置使用 dropout。更完整的第一行是 $\text{LN}_1(\mathbf{X} + \text{Dropout}(\text{MHA}(\mathbf{X})))$ 。本文为突出主线而省略它；这不表示实际训练只有图中最简算式。

7 完整结构

现在可以沿着图 5 从下往上读了。左边处理输入问题，右边处理已知的答案前缀；横跨两边的连线把 Encoder 输出提供给交叉注意力。不同注意力模块之间传递的是特征向量，不是把三张权重矩阵直接拼起来。

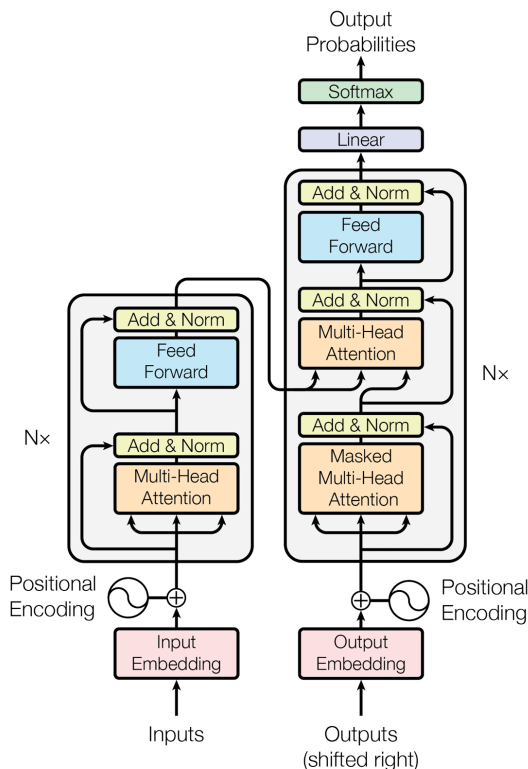


图 5: 原始 Transformer 结构 (原论文 Figure 1)。左半为 Encoder, 右半为 Decoder; 底部的圆圈与 “+” 表示加入位置编码, 绕过各子模块的连线表示残差相加。

7.1 左侧：问题的编码

经过 N 个 Encoder Block, 得到

$$\mathbf{M} = \text{Encoder}(\mathbf{X}^{(0)}) \in \mathbb{R}^{n \times d}. \quad (7.1)$$

各层结构相同, 但参数不同。后一层重新计算自己的 Query、Key、Value 和注意力, 不是重复使用第一层的权重矩阵。每一行仍对应问题中的一个位置, 但已经受到上下文与多层特征变换的影响。

Encoder 并没有在这里直接选出答案，也不要求把整句话压成一个向量。它提供的是一串供 Decoder 读取的连续表示。

7.2 右下：掩码自注意力

第一次生成时，Decoder 的输入只有开始标记 <BOS>。以后则是 <BOS> 加上已经生成的 token。右下角的 Output Embedding，表示把这串已知输出转成向量，不是把向量变成最终文字。

令当前前缀的表示为 $\mathbf{U} \in \mathbb{R}^{t \times d}$ 。在掩码自注意力中， Q, K, V 都来自 $\mathbf{U}$ ，但第 i 个位置只能访问 $j \leq i$ 的位置。用一个加在分数上的矩阵表示规则：

$$C_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad \mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{C}\right). \quad (7.2)$$

对于三个前缀 token， $\mathbf{C}$ 为

$$\begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}.$$

因为 $\exp(-\infty) = 0$ ，不允许访问的未来位置得到零权重。掩码不是把当前输入 token 自己隐藏起来，而是阻止它提前读取后面的 token。

这一模块后面仍有 Add 与 Norm。掩码自注意力的结果已经融入答案前缀内部的信息，但要根据问题作答，还需要下一步。

7.3 右中：交叉注意力

设右边经过前一子模块后得到 $\mathbf{H}$ 。交叉注意力对它和 Encoder 输出分别投影：

$$\mathbf{Q} = \mathbf{H}\mathbf{W}_Q^{\text{cross}}, \quad \mathbf{K} = \mathbf{M}\mathbf{W}_K^{\text{cross}}, \quad \mathbf{V} = \mathbf{M}\mathbf{W}_V^{\text{cross}}. \quad (7.3)$$

Query 来自当前的答案表示；Key、Value 来自问题表示。公式仍然是式 (4.14)，改变的是输入来源。

例如，问题有八个 token，答案前缀有三个 token，每个头使用四维 Query、Key、Value。那么

$$\underbrace{\mathbf{Q}}_{3 \times 4} \underbrace{\mathbf{K}^\top}_{4 \times 8} \longrightarrow \underbrace{\mathbf{A}}_{3 \times 8}, \quad \underbrace{\mathbf{A}}_{3 \times 8} \underbrace{\mathbf{V}}_{8 \times 4} \longrightarrow \underbrace{\mathbf{Z}}_{3 \times 4}. \quad (7.4)$$

这表示答案中的三个位置，分别从问题的八个位置汇总信息。不是把长度八的问题直接加到长度三的答案上。

表 2: 三种注意力的连接关系；矩阵尺寸按单个头计算

模块	Query 的来源	Key、Value 的来源	权重矩阵
Encoder 自注意力	问题的当前表示	同一份问题表示	8×8
Decoder 掩码自注意力	答案前缀的当前表示	同一份前缀表示，屏蔽未来	3×3
Decoder 交叉注意力	右侧前一子模块的输出	Encoder 的最终输出	3×8

7.4 右上：FFN 与 Add & Norm

忽略 dropout，一个 Decoder Block 完整写成

$$\begin{aligned} \mathbf{H} &= \text{LN}_1(\mathbf{U} + \text{MaskedMHA}(\mathbf{U})), \\ \mathbf{R} &= \text{LN}_2(\mathbf{H} + \text{CrossMHA}(\mathbf{H}, \mathbf{M})), \\ \mathbf{D} &= \text{LN}_3(\mathbf{R} + \text{FFN}(\mathbf{R})). \end{aligned} \quad (7.5)$$

三个主要子模块对应三组 Add 与 Norm，每一步都保持右侧 $t \times d$ 的形状。第二行加回的是右侧的 $\mathbf{H}$ ，不是左侧的 $\mathbf{M}$ 。

因此，Decoder 不是“在 Encoder 前面随便多加一个掩码模块”。它把自注意力改成果因果形式，并额外加入读取 Encoder 输出的交叉注意力。下一层 Decoder 继续处理 $\mathbf{D}$ ，同时仍可访问同一份 $\mathbf{M}$ ；各层使用自己的投影参数。

8 逐步生成

8.1 一个完整例子

假设任务输入是“谁是第一个登上月球的人？”，期望简短回答“阿姆斯特朗”。第一位登上月球的人是 Neil A. Armstrong。这里假定模型已经接受相关训练；未经训练的网络不会仅凭框架图自动知道这条事实。

人为把问题分为八个 token：

[谁, 是, 第一个, 登上, 月球, 的, 人, ?].

这只是教学切分。Encoder 产生 $\mathbf{M} \in \mathbb{R}^{8 \times 12}$ ；答案为了展示按单字生成。

假设当前前缀是“<BOS> 阿姆”，它有三个 token。右边完成式 (7.5) 并经过全部 Decoder 层，得到一个 3×12 的矩阵。每行都是对应位置的上下文表示。生成下一个 token 时，取最后一行，也就是当前“姆”所在位置的向量：

$$\mathbf{d}_{\text{last}} \in \mathbb{R}^{1 \times 12}. \quad (8.1)$$

取最后一行并不表示忽略前面位置；这个向量已经通过注意力读取了允许访问的前缀和问题信息。

8.2 Linear 与 Softmax

假设目标词表有 10,000 个 token。输出线性层采用

$$\mathbf{W}_{\text{vocab}} \in \mathbb{R}^{12 \times 10000}, \quad \boldsymbol{\ell} = \mathbf{d}_{\text{last}} \mathbf{W}_{\text{vocab}} + \mathbf{b} \in \mathbb{R}^{1 \times 10000}. \quad (8.2)$$

这里每个分数对应词表中的一个候选 token，而不是一个输入位置。再计算

$$P(y_t = j \mid s, y_{<t}) = \frac{\exp(\ell_j)}{\sum_{r=1}^{|\mathcal{V}_{\text{out}}|} \exp(\ell_r)}. \quad (8.3)$$

于是得到了下一个 token 的概率分布。

这和注意力里的 softmax 是同一种数学运算，但对象不同：注意力在“信息来源的位置”之间分配权重，输出层在“词表候选”之间分配概率。

softmax 本身不负责选中某个 token；还需要解码规则，例如取概率最高者，或按照分布采样。

为展示最简单的流程，本文采用逐步取最大概率的贪心选择：

$$\hat{y}_t = \arg \max_j P(y_t = j \mid s, \hat{y}_{<t}).$$

它是一个演示选择，不意味着训练后的模型必然给正确答案最高概率，也不保证贪心选择得到全序列概率最大的结果。

8.3 生成循环

假设每一步恰好选出期望的字符，整个过程如表 3 所示。

表 3: 一次假设成功的生成轨迹；token 切分与逐步结果仅作教学演示

步骤	右边已经知道的输入前缀	新选出的 token
1	<BOS>	阿
2	<BOS> 阿	姆
3	<BOS> 阿姆	斯
4	<BOS> 阿姆斯	特
5	<BOS> 阿姆斯特	朗
6	<BOS> 阿姆斯特朗	<EOS>

选到结束标记 <EOS> 后停止，去掉特殊标记，显示“阿姆斯特朗”。实际也可设置最大生成长度，避免一直不结束。

这解释了“并行计算”和“逐步生成”为什么并不矛盾：对于已知前缀，同一层的各位置可以一起计算；但新的 token 尚未被选出，下一次前缀就还没有确定。实际系统可以缓存已经算过的 Key、Value 来避免部分重复工作，这不改变这里的逻辑。

答案中的人名没有出现在输入问题里，并不妨碍输出它，因为顶部是在整个目标词表上打分，不是只能复制源句子的词。至于哪些候选应当获

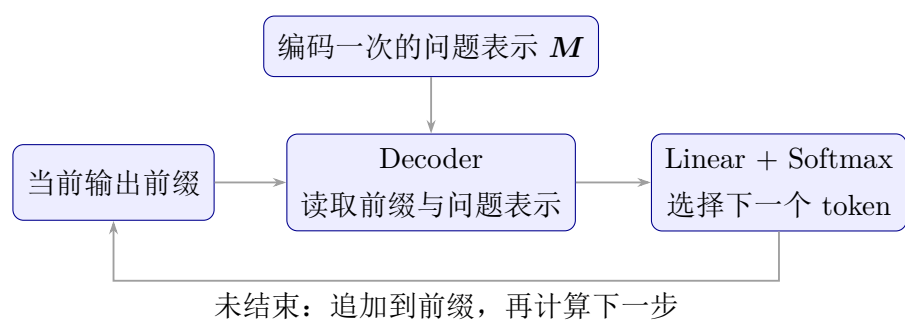


图 6: 生成中的循环。左边 Encoder 的结果可以复用；模型参数在正常推理中不更新。

得高分，要由训练形成的参数与当前条件共同决定；这种机制本身没有事实正确性的保证。

9 训练目标

9.1 Outputs shifted right

训练时已经拥有问题和正确答案。原图右下角的 “Outputs shifted right”，说的是怎么把正确答案变成 Decoder 的输入：在开头补 <BOS>，让输入与预测目标错开一位。

表 4: 训练时的输入与目标对齐；每列练习 “根据已知前缀预测下一个 token”。

位置	1	2	3	4	5	6
Decoder 输入	<BOS>	阿	姆	斯	特	朗
应该预测	阿	姆	斯	特	朗	<EOS>

例如输入 “姆” 的位置，目标是预测 “斯”。虽然训练矩阵中右边也放着 “斯”，因果掩码阻止这个位置读取它。模型没有偷看当前要预测的答案。

这类使用真实前缀进行训练的方式常称为 teacher forcing。完整答案已知，使不同目标位置的损失能够在一次前向中一起计算；生成时则没有真实前缀可用，只能追加自己已经选出的 token。

9.2 预测损失

把源句记为 s ，目标序列 $y = (y_1, \dots, y_m)$ 包含结束标记。自回归形式为

$$p_\theta(y | s) = \prod_{i=1}^m p_\theta(y_i | s, y_{<i}). \quad (9.1)$$

让训练答案获得更高概率，可以等价地最小化负对数似然：

$$\mathcal{L}(\theta) = -\frac{1}{m} \sum_{i=1}^m \log p_\theta(y_i | s, y_{<i}). \quad (9.2)$$

这里给出未使用标签平滑、未考虑 padding 的基本形式；实际批量训练需忽略补齐位置，原论文还使用了标签平滑。

如果某一位置正确 token 的预测概率从 0.1 提高到 0.8，这一位置的损失就从 $-\log 0.1 \approx 2.303$ 降到 $-\log 0.8 \approx 0.223$ 。训练在鼓励正确预测，不是在把所有激活值或所有残差更新压向零。

θ 包括输入和输出的嵌入、各头的投影、输出混合矩阵、FFN、LN 的缩放和平移、词表输出层等参数。原论文在适用的共享词表设置中还共享了部分嵌入和输出权重；为了把数据流讲清楚，本文没有要求这些符号必须相互独立。

表 5: 可训练参数与当前输入产生的中间结果

类别	例子	它们如何变化
可训练参数	嵌入表、各类 $\mathbf{W}$ 、偏置、 γ 和 β	根据训练损失更新；正常推理时固定
中间结果	$\mathbf{X}, \mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{A}, \mathbf{Z}, \mathbf{M}$	每次随当前输入和参数重新计算
固定规则	正弦位置编码、因果掩码、选定的模型尺寸	在本文的设定中不是梯度优化的参数

因此，“训练注意力”并不是直接存下一张永不变化的关联表，而是学习产生投影与权重的参数。最终损失沿着完整计算图反向传播，包括注意

力、Add、Norm、FFN 和嵌入。没有人提前告诉每一层的向量应该等于什么数值。

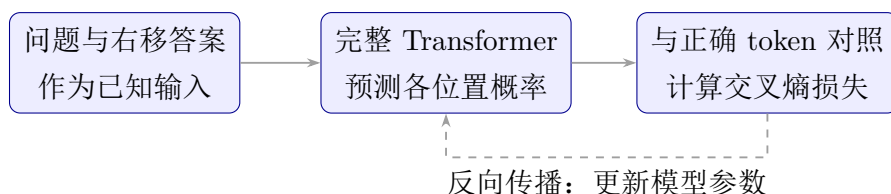


图 7: 训练的数据流。完整网络包含可训练的词嵌入；输入的整数 token 编号本身不作为连续参数优化。

10 小结

现在再看 Transformer，已经不必靠“某个模块负责理解，另一个模块负责思考”这样的比喻来记忆。它始终在处理一串向量：嵌入把离散编号变成特征，位置编码引入顺序；注意力根据匹配关系汇总不同位置的信息，多头保留多套汇总方式；残差把模块组织成对已有表示的更新，归一化控制尺度，FFN 在每个位置内部进一步构造非线性特征。

左侧将问题编码成 M ，右侧根据已有答案前缀构造 Query，通过交叉注意力读取 M 。最后一个位置的向量被映射到词表分数，再变成下一 token 的概率；选出的 token 被追加到前缀，直到结束。训练则用正确答案的概率定义损失，将这整条链上的参数一起调整。

$$\boxed{\text{构造表示} \rightarrow \text{交换与变换信息} \rightarrow \text{输出条件概率}} \quad (10.1)$$

最值得记住的不是那张图里有多少个框，而是每个框改变了什么。 A 是信息分配比例，不是最终特征；Add 是模型定义的一部分，不是训练后的额外补丁；LN 接收的是前一步计算的结果，不要求括号里永远只有一个字母；FFN 的作用来自可学习的非线性变换，而不是“维度升高就更干净”。

“Transformer”也并不强制所有模型都必须保留原图中的两座塔。这里讨论的是原始编码器—解码器版本；理解它的模块和数据流以后，再看使用其中部分结构的模型，就可以逐项比较它保留了哪些连接、修改了哪些约束，而不必重新从术语开始。